DevDas — Autonomous Kubernetes SRE Investigation Agent

Project Profile Document · AMD AI Hackathon Submission

Executive Summary

DevDas is an autonomous AI agent that investigates unhealthy Kubernetes workloads, identifies root causes, and generates actionable recovery plans — without human intervention. The system combines a pure-Python cluster discovery engine with an LLM-driven reasoning loop (Qwen 2.5-3B-Instruct) that selects and executes Kubernetes diagnostic tools one at a time, accumulating evidence until a confident root cause determination is reached. A hybrid deployment architecture separates GPU-accelerated AI inference (running in an AMD notebook environment) from local Kubernetes tool execution (exposed via a FastAPI server and ngrok tunnel), demonstrating a modular and scalable approach to AI-powered infrastructure operations.

Table of Contents

1. Project Overview
2. Project Background
3. Target Users
4. Application Scenarios
5. System Architecture
6. Agent Workflow
7. Core Capabilities
8. Model & Algorithm
9. Supported Failure Scenarios
10. AMD Radeon GPU Adaptation
11. Future Work
12. Conclusion

---

1. Project Overview

1.1 Project Name

DevDas — Autonomous Kubernetes SRE Investigation Agent

1.2 Abstract

Kubernetes cluster failures remain one of the most time-consuming operational challenges in modern cloud infrastructure. DevDas addresses this by implementing a fully autonomous Site Reliability Engineering (SRE) investigation agent. Given a Kubernetes cluster, DevDas automatically discovers unhealthy workloads, conducts systematic evidence-driven investigations using 17 purpose-built diagnostic tools, and produces structured root cause analyses with concrete recovery plans. The system's reasoning engine — powered by the Qwen 2.5-3B-Instruct language model running on AMD GPU hardware — emulates the step-by-step diagnostic methodology of an experienced SRE, while deterministic pattern-matching accelerators provide instant resolution for well-known failure signatures.

1.3 Problem Statement

Kubernetes environments are inherently complex, comprising multiple interdependent resource types (Pods, Deployments, Services, ConfigMaps, Secrets, Nodes) across multiple namespaces. When failures occur, SREs must manually inspect pod statuses, read logs, correlate events, check configurations, verify service connectivity, and reason about the interaction of all these components. This process is:

- **Slow**: A single failure investigation can consume 30–90 minutes of senior engineer time.
- **Error-prone**: Manual correlation across dozens of kubectl outputs invites oversight.
- **Expensive**: Senior SRE time is a scarce and costly resource.

- **Unrepeatable**: Institutional knowledge is lost when personnel rotate.

1.4 Motivation

The operational burden of Kubernetes troubleshooting scales non-linearly with cluster size and microservice count. As organizations adopt Kubernetes at scale, the gap between incident volume and available SRE capacity widens. DevDas was motivated by the vision of an AI agent that can conduct first-pass investigations autonomously, reducing mean time to resolution (MTTR) and freeing human engineers for higher-order architectural work.

1.5 Objectives

#	Objective	Status
1	Autonomously discover all unhealthy workloads across a Kubernetes cluster	✅ Implemented
2	Conduct evidence-driven, step-by-step root cause investigations	✅ Implemented
3	Execute one diagnostic tool at a time with structured reasoning	✅ Implemented
4	Generate actionable recovery plans for confirmed root causes	✅ Implemented
5	Produce structured Cluster Investigation Reports	✅ Implemented
6	Run AI inference on AMD GPU hardware via Qwen 2.5-3B-Instruct	✅ Implemented
7	Support hybrid deployment (GPU inference + local K8s tools)	✅ Implemented

1.6 Key Contributions

1. **Autonomous Investigation Loop**: A novel LLM-driven reasoning cycle that accumulates evidence one tool at a time, mirrors SRE cognitive processes, and terminates deterministically.
2. **Hybrid Inference-Execution Architecture**: Clean separation of GPU-accelerated model inference from CPU-bound Kubernetes diagnostic tool execution, connected via FastAPI and ngrok.
3. **Deterministic Pattern Matching**: Regex-based failure signature detection that provides instant, guaranteed-correct diagnoses for well-known Kubernetes failure modes, bypassing the LLM entirely when evidence is unambiguous.
4. **17 Purpose-Built Diagnostic Tools**: A comprehensive kubectl-backed tool library spanning pods, deployments, services, ConfigMaps, Secrets, nodes, namespaces, PVCs, quotas, and autoscalers.
5. **Structured Output Protocol**: A strict JSON-based communication contract between the orchestrator and the LLM, ensuring parseable, actionable outputs at every iteration.

---

2. Project Background

2.1 Why Kubernetes Troubleshooting Is Difficult

Kubernetes abstracts infrastructure into a declarative resource model with dozens of interdependent object types. A single application failure can originate at any layer:

- **Container level**: Image pull failures, OOMKilled, application crashes
- **Pod level**: Scheduling failures, init container errors, readiness probe failures
- **Configuration level**: Missing ConfigMaps, absent Secrets, malformed environment variables
- **Service level**: Selector mismatches, missing endpoints, DNS resolution failures
- **Node level**: Resource exhaustion, taints/tolerations mismatches, node NotReady
- **Cluster level**: Quota exhaustion, RBAC misconfiguration, network policy conflicts

An SRE investigating a `CrashLoopBackOff` must determine whether the issue is an application bug, a missing configuration dependency, an infrastructure constraint, or a combination thereof. This requires correlating data from multiple kubectl commands and reasoning about causal chains.

2.2 Why Manual Debugging Is Expensive

Metric	Manual Investigation	DevDas
Average investigation time	30–90 minutes	1–5 minutes
Required expertise level	Senior SRE	Automated
Consistency of diagnosis	Variable	Deterministic for known patterns
Knowledge retention	Personnel-dependent	Codified in tools and patterns
Off-hours coverage	Requires on-call staffing	Autonomous

2.3 Why AI Agents Are Suitable for Autonomous SRE

Traditional rule-based monitoring systems (e.g., Prometheus alerts) detect symptoms but do not diagnose causes. LLM-powered agents bridge this gap because they can:

1. **Reason about unstructured data**: Parse and understand kubectl output, application logs, and event streams.
2. **Form and test hypotheses**: Systematically narrow down failure causes through evidence gathering.
3. **Generalize across failure modes**: Handle novel combinations of symptoms without pre-programmed rules for every edge case.
4. **Produce human-readable explanations**: Generate reasoning traces and recovery plans that engineers can audit and act on.

2.4 How DevDas Addresses These Problems

DevDas combines the strengths of deterministic engineering (pattern matching, tool orchestration, structured outputs) with LLM-based reasoning to create a system that is both reliable and adaptive:

- **Deterministic where possible**: Known failure signatures (missing ConfigMap, ErrImageNeverPull, etc.) are matched via regex and resolved instantly.
- **Intelligent where necessary**: Novel or ambiguous failures are investigated through the LLM reasoning loop.
- **Bounded always**: Max iteration limits, duplicate tool guards, and forced final diagnosis mechanisms ensure the agent always terminates.

---

3. Target Users

User Segment	Use Case
DevOps Engineers	Automated first-pass investigation of deployment failures in CI/CD pipelines
Site Reliability Engineers	Accelerated incident response with structured root cause analysis
Platform Engineers	Continuous cluster health monitoring and diagnosis
Cloud Operations Teams	Autonomous off-hours incident investigation
Students Learning Kubernetes	Educational tool demonstrating SRE investigation methodology
Enterprises Operating K8s Clusters	Reduced MTTR and operational cost at scale

---

4. Application Scenarios

4.1 Production Outage Investigation

When a production service enters `CrashLoopBackOff`, DevDas autonomously discovers the unhealthy pod, traces the

failure through events, logs, and deployment configuration, and produces a root cause determination with recovery steps — before a human engineer has opened their terminal.

4.2 Failed Kubernetes Deployments

After a `kubectl apply` results in pods stuck in `CreateContainerConfigError`, DevDas identifies the missing ConfigMap or Secret reference and recommends the exact manifest to recreate.

4.3 Configuration Debugging

When a deployment references a ConfigMap containing an incorrect `DATABASE_URL`, DevDas follows the evidence chain: pod logs show connection failures → deployment spec reveals the referenced ConfigMap → ConfigMap data exposes the malformed URL → root cause confirmed.

4.4 Incident Response

DevDas serves as an automated first responder. During an incident, the agent produces a structured investigation report that on-call engineers can review to accelerate remediation, rather than starting investigations from scratch.

4.5 CI/CD Validation

Integrated into a deployment pipeline, DevDas can scan a target namespace after each deployment and flag any unhealthy workloads before they affect production traffic.

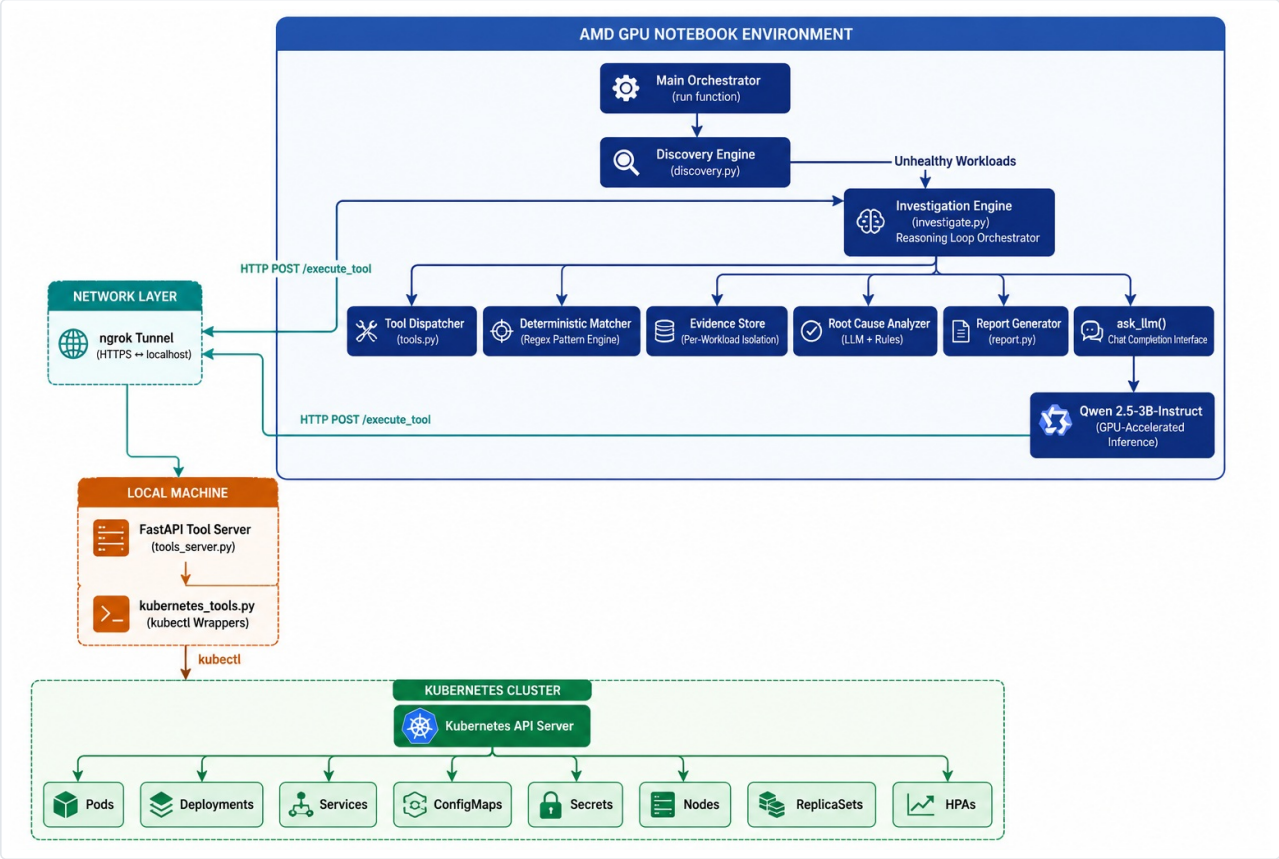
4.6 Cluster Health Diagnosis

Running in `--all-namespaces` mode, DevDas performs a complete cluster health scan, investigating every unhealthy workload and generating a consolidated Cluster Investigation Report.

---

5. System Architecture

5.1 Architecture Diagram



Hybrid architecture diagram showing the AMD GPU Notebook environment, network layer, local machine, and Kubernetes cluster

5.2 Architectural Components

Component	Responsibility	Execution Environment
Qwen 2.5-3B-Instruct	LLM reasoning — hypothesis generation, tool selection, root cause determination	AMD GPU (Notebook)
ask_llm()	Chat completion interface wrapping tokenizer + model.generate()	AMD GPU (Notebook)
System Prompt	SRE persona definition, tool schema, golden rules, triage guide, output schema	AMD GPU (Notebook)
Investigation Orchestrator	Reasoning loop: evidence → LLM → tool → evidence → repeat	AMD GPU (Notebook)
Deterministic Matcher	Regex-based instant resolution for known failure signatures	AMD GPU (Notebook, CPU)
Evidence Store	Per-workload isolated dictionary for collected diagnostic data	AMD GPU (Notebook, CPU)
Tool Dispatcher	Routes LLM tool selections to the correct kubernetes_tools function	AMD GPU (Notebook, CPU)
Discovery Engine	Pure-Python cluster scan to identify all unhealthy workloads	AMD GPU (Notebook, CPU)
Report Generator	Formats investigation results into structured Cluster Investigation Reports	AMD GPU (Notebook, CPU)
ngrok Tunnel	Secure HTTPS tunnel connecting notebook to local machine	Network
FastAPI Tool Server	HTTP endpoint that receives tool execution requests and dispatches them	Local Machine (CPU)
kubernetes_tools.py	17 kubectl subprocess wrappers that execute against the live cluster	Local Machine (CPU)
Kubernetes Cluster	The target cluster being investigated	Infrastructure

5.3 Data Flow

1. The **Main Orchestrator** initiates a cluster scan via the **Discovery Engine**.

2. Discovery calls `discover_unhealthy_workloads()` through the **ngrok tunnel** to the **FastAPI Tool Server**, which runs `kubectl get pods -o json` against the live cluster.

3. Returned pod data is analyzed client-side: unhealthy pods are identified by phase, container state reasons (`CrashLoopBackOff`, `ImagePullBackOff`, `CreateContainerConfigError`, etc.), and restart thresholds.

4. Each unhealthy workload is passed to the **Investigation Orchestrator**, which initializes an isolated **Evidence Store** and enters the **Reasoning Loop**.

5. At each iteration, the orchestrator composes a prompt (workload description + collected evidence + remaining tools) and queries the **Qwen model** via `ask_llm()`.

6. The LLM returns structured JSON: either `status: "need_more_evidence"` with a tool request, or `status: "complete"` with a root cause and recovery plan.

7. If the LLM requests a tool, the **Tool Dispatcher** forwards the request through ngrok to the **FastAPI Tool Server**, which executes the corresponding kubectl command and returns the result.

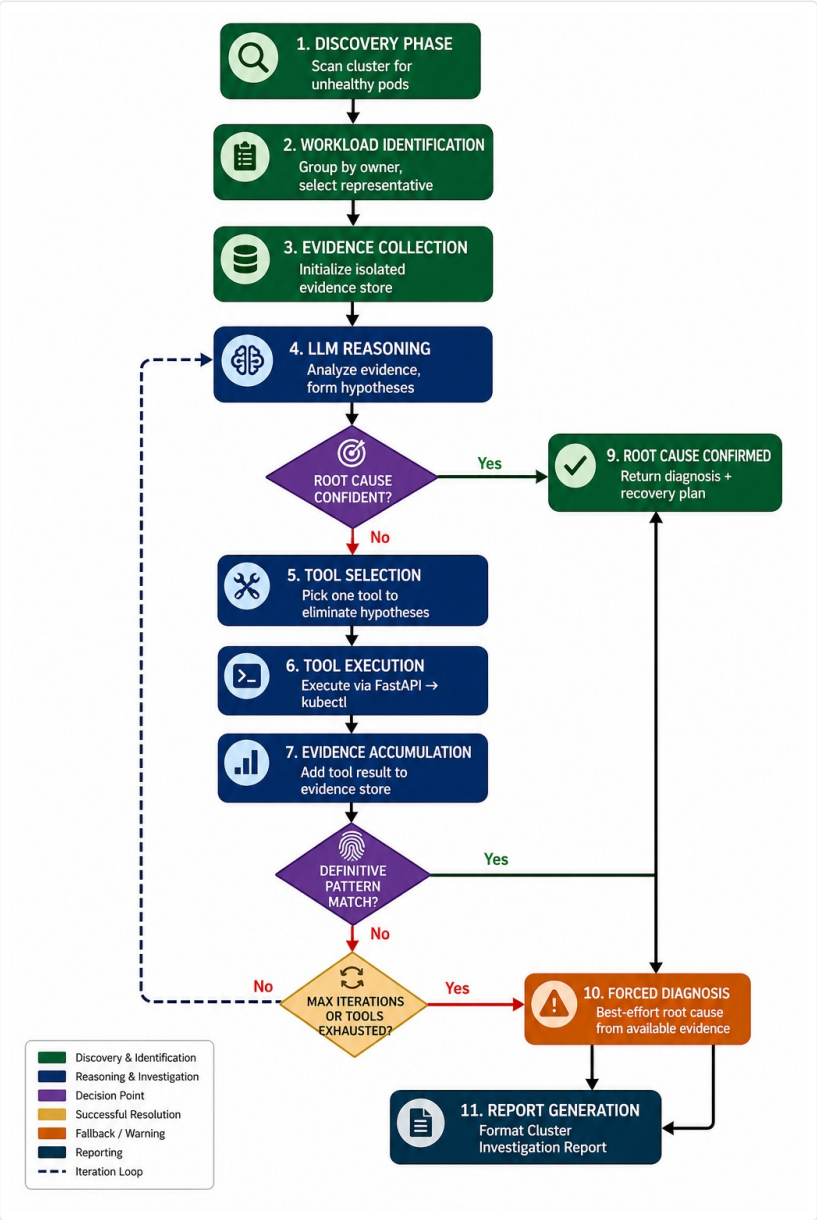
8. After each tool execution, the **Deterministic Matcher** scans all evidence for known failure signatures. If a match is found, the investigation short-circuits with an instant diagnosis.

9. Once all workloads are investigated, the **Report Generator** produces a consolidated Cluster Investigation Report.

---

6. Agent Workflow

6.1 End-to-End Pipeline



End-to-end pipeline flowchart showing the eleven-step investigation process from discovery to report generation

6.2 Phase Descriptions

Phase 1: Discovery The discovery engine runs `kubectl get pods -o json` across all non-system namespaces. Each pod is evaluated against a set of unhealthy conditions:

- Pod phase ∈ { `Failed`, `Unknown`, `Pending` }
- Container waiting reason ∈ { `CrashLoopBackOff`, `ImagePullBackOff`, `ErrImagePull`, `OOMKilled`, `Error`, `CreateContainerConfigError`, `InvalidImageName`, `RunContainerError`, `ContainerCannotRun`, `DeadlineExceeded` }
- Container terminated reason ∈ { `OOMKilled`, `Error`, `ContainerCannotRun` }
- Restart count ≥ 5 (high restart threshold)

System namespaces (`kube-system`, `kube-public`, `kube-node-lease`) are excluded.

Phase 2: Investigation Each unhealthy workload enters the reasoning loop. The orchestrator resolves the owning Deployment name (walking Pod → ReplicaSet → Deployment), initializes an evidence store, and begins iterating:

1. Compose a prompt containing the incident description, all collected evidence, investigation summary, and

remaining available tools.

2. Query the Qwen model for a JSON response.
3. Parse and validate the response.
4. If `status: "complete"` → return the diagnosis.
5. If `status: "need_more_evidence"` → validate the requested tool, execute it, add results to evidence, check for deterministic pattern matches.
6. Repeat until completion, max iterations (20), or tool exhaustion.

Phase 3: Report Generation All investigation results are aggregated into a Cluster Investigation Report containing: - Cluster summary statistics (total pods, unhealthy count, tool calls, duration) - Per-workload investigation details (root cause, recovery plan, tools used, iterations, duration)

---

7. Core Capabilities

7.1 Autonomous Investigation

DevDas requires zero human input after launch. The agent discovers problems, decides what to investigate, selects which tools to use, interprets results, and produces diagnoses — all autonomously. The system prompt encodes the persona of "an autonomous Kubernetes SRE investigation agent" whose job is strictly: *"Why is this workload unhealthy right now?"*

7.2 One-Tool-at-a-Time Reasoning

A fundamental design constraint enforced by the system prompt: the LLM must request **exactly one tool per response**. This ensures: - Each tool's output is fully processed before the next decision. - The evidence chain is traceable and auditable. - Token budgets are managed by limiting context growth to one tool result per iteration.

7.3 Dynamic Evidence Gathering

Evidence is accumulated in a per-workload isolated dictionary. The dictionary starts empty and grows dynamically as tools execute. Each tool's output is keyed by tool name, enabling the LLM to reference specific pieces of evidence in its reasoning. Evidence is never shared between workload investigations, preventing cross-contamination.

7.4 Tool Orchestration

The tool dispatcher (`tools.py`) routes LLM tool requests to the corresponding `kubernetes_tools` function via a structured dispatch table. The dispatcher supports 17 tools spanning six categories:

Category	Tools
Pod Tools	`get_pod_logs`, `get_pod_events`, `describe_pod`
Deployment Tools	`describe_deployment`, `get_deployment_yaml`, `get_replicaset_status`
Service Tools	`describe_service`, `get_endpoints`
Configuration Tools	`get_configmap`, `get_secret`
Node Tools	`get_nodes`, `describe_node`
Cluster Tools	`get_namespace_resources`, `get_persistent_volume_claims`, `get_resource_quota`, `get_all_pods`, `get_horizontal_pod_autoscaler`

7.5 Deterministic Stopping

The investigation loop employs multiple termination mechanisms:

1. **LLM-driven completion**: The model returns `status: "complete"` when evidence uniquely points to one root cause.
2. **Deterministic pattern matching**: Regex patterns detect definitive failure signatures immediately after each tool execution.
3. **Max iteration guard**: After 20 iterations, the agent is forced to produce a best-effort diagnosis.

4. **Tool exhaustion**: When all available tools have been used, the agent is forced to conclude.
5. **Duplicate tool guard**: Repeated requests for already-used tools are rejected, with remaining tools surfaced to the LLM.

7.6 Root Cause Analysis

Root causes are identified through two complementary mechanisms:

- **LLM Reasoning**: The model synthesizes evidence across multiple tool outputs to form and eliminate hypotheses, producing a root cause statement with supporting reasoning.
- **Deterministic Detection**: The `DEFINITIVE_PATTERNS` list contains regex patterns for seven well-known Kubernetes failure signatures. When matched, the root cause is immediately confirmed without further LLM involvement.

7.7 Recovery Recommendation

Every confirmed root cause is accompanied by an actionable recovery plan. For deterministically matched failures, recovery plans are generated from a curated lookup table (`_recovery_plan_for()`). For LLM-diagnosed failures, the model generates recovery steps as part of its completion response.

7.8 Investigation Report Generation

The report generator produces a structured Cluster Investigation Report containing:

- Cluster-level summary statistics
- Per-workload investigation results with root causes, recovery plans, tools used, iteration counts, and durations
- Investigation status badges (✅ `complete` or ⚠ `complete_forced`)
- Optional file-based export with ANSI code stripping for clean text output

---

8. Model & Algorithm

8.1 Discovery Algorithm

```
FUNCTION discover_unhealthy_workloads(namespace):
    pods ← kubectl_get_pods_json(namespace)
    unhealthy ← []

    FOR EACH pod IN pods:
        IF pod.namespace IN {kube-system, kube-public, kube-node-lease}:
            SKIP

        problem_reason ← NULL
        restart_count ← max(container.restartCount for all containers)

        FOR EACH container_status IN pod.containerStatuses:
            IF container.state.waiting.reason IN UNHEALTHY_REASONS:
                problem_reason ← reason
                BREAK
            IF container.state.terminated.reason IN {OOMKilled, Error, ContainerCannotRun}:
                problem_reason ← reason
                BREAK

        is_unhealthy ← FALSE
        IF problem_reason:
            is_unhealthy ← TRUE
        ELSE IF pod.phase IN {Failed, Unknown, Pending}:
            is_unhealthy ← TRUE
        ELSE IF restart_count ≥ 5:
            is_unhealthy ← TRUE

        IF is_unhealthy:
            unhealthy.append({pod_name, namespace, status, restart_count,
                              node, owner_kind, owner_name})

    RETURN unhealthy, total_pod_count
```

8.2 Investigation Loop

```
FUNCTION investigate(workload, system_prompt):
    evidence ← {}
    used_tools ← {}
    deployment_name ← resolve_pod_to_deployment(workload)
    iteration ← 0

    WHILE iteration < MAX_ITERATIONS (20):
        iteration += 1

        remaining_tools ← AVAILABLE_TOOLS \ used_tools
        prompt ← compose_prompt(workload, evidence, remaining_tools)

        response ← ask_llm(system_prompt, prompt)
        data ← parse_json(response)

        IF data.status == "complete":
            RETURN {root_cause, recovery_plan, reasoning, tools_used}

        tool ← data.next_action.tool

        IF tool NOT IN AVAILABLE_TOOLS:
            evidence["SYSTEM_ERROR"] ← "Invalid tool"
            CONTINUE

        IF tool IN used_tools:
            IF remaining_tools IS EMPTY:
                RETURN force_final_diagnosis(evidence)
            evidence["SYSTEM_WARNING"] ← "Tool already used"
            CONTINUE

        result ← execute_tool(tool, args)
        used_tools.add(tool)
        evidence[tool] ← result

        definitive ← detect_definitive_cause(evidence)
        IF definitive IS NOT NULL:
            RETURN {root_cause: definitive, recovery_plan: lookup(definitive)}

    RETURN force_final_diagnosis(evidence)
```

8.3 Evidence Store Design

The evidence store is implemented as a factory function (`new_evidence()`) that returns a fresh, empty Python dictionary. Keys are added dynamically as tools execute — there is no pre-defined schema. This design provides:

- **Per-workload isolation**: Each investigation gets its own evidence store.
- **Dynamic extensibility**: New tool outputs are accommodated without schema changes.
- **Simple serialization**: The dictionary is directly serializable to JSON for inclusion in LLM prompts.

8.4 Tool Dispatcher Architecture

```
LLM Response (JSON)
    │
    ├─ tool: "get_pod_events"
    │  arguments: {pod_name: "X", namespace: "Y"}
    │
    ▼
execute_tool(tool_name, args, evidence)
    │
    ├─ _dispatch(tool_name, args)
    │    │
```

```
    │    ├─ get_pod_events(pod_name, namespace)
    │    │    │
    │    │    └─ _call_remote_tool("get_pod_events", ...)
    │    │         │
    │    │         └─ HTTP POST → ngrok → FastAPI → kubectl
    │    │
    │    └─ return result
    │
    └─ evidence["get_pod_events"] = result
```

8.5 LLM Reasoning Cycle

The LLM operates within a strict structured output protocol. At each iteration, it receives:

1. **Workload description**: Pod name, namespace, status, restart count, owner, deployment name.
2. **Collected evidence**: All tool outputs accumulated so far.
3. **Investigation summary**: List of tools used and evidence keys collected.
4. **Remaining tools**: Descriptions of tools not yet used.

The model must return one of two JSON responses:

Need More Evidence:

```
{
    "status": "need_more_evidence",
    "reasoning": "...",
    "current_hypotheses": ["Hypothesis 1", "Hypothesis 2"],
    "next_action": {
        "tool": "<tool_name>",
        "arguments": { ... }
    },
    "root_cause": null,
    "recovery_plan": []
}
```

Investigation Complete:

```
{
    "status": "complete",
    "reasoning": "...",
    "current_hypotheses": ["Confirmed root cause"],
    "next_action": null,
    "root_cause": "One-sentence root cause.",
    "recovery_plan": ["Step 1: ...", "Step 2: ..."]
}
```

8.6 Deterministic Signature Matching

The `DEFINITIVE_PATTERNS` engine runs after each tool execution. It scans all string-valued evidence entries against seven regex patterns:

Pattern	Matched Root Cause
`configmap "<name>" not found`	Missing ConfigMap: `<name>` does not exist
`secret "<name>" not found`	Missing Secret: `<name>` does not exist
`ErrImageNeverPull`	Image pull policy misconfiguration
`ImagePullBackOff`	Image cannot be pulled — check image name/tag/registry
`ErrImagePull`	Image pull failure — check image name/tag/registry
`FailedScheduling.*Insufficient (cpu|memory)`	Pod unschedulable — insufficient resources
`FailedScheduling.*node(s) had`	Pod unschedulable — taint/toleration mismatch

`FailedScheduling: node(s) had taint`	Pod unschedulable — taint/toleration mismatch

When a pattern matches, the investigation immediately returns with the matched root cause and a pre-built recovery plan, bypassing any further LLM calls. This provides guaranteed correctness and near-zero latency for the most common Kubernetes failure modes.

8.7 Investigation Stopping Criteria

The system prompt encodes five explicit stopping rules for the LLM:

1. One hypothesis explains **all** collected evidence.
2. Competing hypotheses have been **eliminated** by evidence.
3. A production SRE could **act** on the diagnosis immediately.
4. All remaining unused tools would return **redundant** information.
5. Tools are failing (`SYSTEM_ERROR`) and no alternative tool exists.

The orchestrator additionally enforces three programmatic stops:

1. **Max iterations**: 20 iterations → forced final diagnosis.
2. **Tool exhaustion**: All 17 tools used → forced final diagnosis.
3. **Deterministic match**: Regex pattern hit → immediate return.

---

9. Supported Failure Scenarios

The DevDas project includes a suite of five reproducible failure scenarios, each implemented as a pair of batch scripts (`break_*.bat` / `fix_*.bat`) operating on a reference Kubernetes deployment (payment-service + PostgreSQL in the `devdas` namespace).

9.1 Missing ConfigMap

Attribute	Detail
Injection Method	`kubectl delete configmap payment-config -n devdas` followed by deployment rollout restart
Expected Pod Status	`CreateContainerConfigError`
Failure Symptoms	Pod enters `Pending` → container creation fails because `envFrom.configMapRef` references `payment-config`, which no longer exists
Investigation Path	`get_pod_events` → event shows `configmap "payment-config" not found` → deterministic pattern match fires
Root Cause	Missing ConfigMap: `payment-config` does not exist in namespace `devdas`
Recovery Steps	1. Identify the correct ConfigMap manifest. 2. `kubectl apply -f payment-configmap.yaml`. 3. Verify pod reaches Running.

9.2 Missing Secret

Attribute	Detail
Injection Method	`kubectl delete secret payment-secret -n devdas` followed by deployment rollout restart
Expected Pod Status	`CreateContainerConfigError`
Failure Symptoms	Container creation fails because `envFrom.secretRef` references `payment-secret`, which no longer exists

Investigation Path	`get_pod_events` → event shows `secret "payment-secret" not found` → deterministic pattern match fires
Root Cause	Missing Secret: `payment-secret` does not exist in namespace `devdas`
Recovery Steps	1. Identify the correct Secret source (Vault, sealed-secret, or CI/CD). 2. Recreate the Secret. 3. Verify pod reaches Running.

9.3 Wrong DATABASE_URL

Attribute	Detail
Injection Method	Delete and recreate `payment-config` with `DATABASE_URL=postgresql://admin:password@does-not-exist:5432/postgres`
Expected Pod Status	`CrashLoopBackOff`
Failure Symptoms	Application starts, reads `DATABASE_URL` from ConfigMap, attempts PostgreSQL connection to non-existent host, `psycopg2.connect()` raises an exception, container exits with error, Kubernetes restarts it repeatedly
Investigation Path	`get_pod_logs(previous=true)` → connection error in logs → `describe_deployment` → reveals referenced ConfigMap → `get_configmap` → exposes malformed `DATABASE_URL` → LLM correlates evidence
Root Cause	`DATABASE_URL` in ConfigMap `payment-config` points to a non-existent PostgreSQL host
Recovery Steps	1. Correct the `DATABASE_URL` in the ConfigMap. 2. Apply the corrected ConfigMap. 3. Rollout restart the deployment.

9.4 PostgreSQL Down

Attribute	Detail
Injection Method	`kubectl scale deployment postgres --replicas=0 -n devdas` followed by payment-service rollout restart
Expected Pod Status	`CrashLoopBackOff`
Failure Symptoms	PostgreSQL pod is scaled to zero → payment-service starts, reads the correct `DATABASE_URL`, but `psycopg2.connect()` cannot reach the PostgreSQL service → connection timeout/refusal → container crash loop
Investigation Path	`get_pod_logs(previous=true)` → connection refused errors → `get_all_pods` or `describe_service` → PostgreSQL pod not running → LLM determines dependency failure
Root Cause	PostgreSQL dependency is unavailable (zero replicas)
Recovery Steps	1. Scale PostgreSQL back to 1+ replicas: `kubectl`

	`scale deployment postgres --replicas=1` . 2. Wait for PostgreSQL to reach Running. 3. Rollout restart payment-service.

9.5 Invalid Image / ErrImageNeverPull

Attribute	Detail
Injection Method	`kubectl set image deployment/payment-service payment-service=payment-service:this-tag-does-not-exist -n devdas`
Expected Pod Status	`ErrImagePull` / `ImagePullBackOff` / `ErrImageNeverPull`
Failure Symptoms	Pod enters image pull failure state because the specified image tag does not exist locally and `imagePullPolicy: Never` prevents pulling from a registry
Investigation Path	`get_pod_events` → events show `ErrImageNeverPull` or `ImagePullBackOff` → deterministic pattern match fires
Root Cause	Image pull policy misconfiguration — the specified image tag does not exist locally
Recovery Steps	1. Confirm the image exists locally on the node, or 2. Set `imagePullPolicy` to `IfNotPresent` / `Always` and ensure registry access is configured.

---

10. AMD Radeon GPU Adaptation

10.1 Notebook Architecture

The AMD AI Hackathon submission runs as a **Jupyter notebook** on AMD GPU hardware. The notebook replaces the original Groq API client with a locally-loaded **Qwen 2.5-3B-Instruct** model using Hugging Face `transformers`:

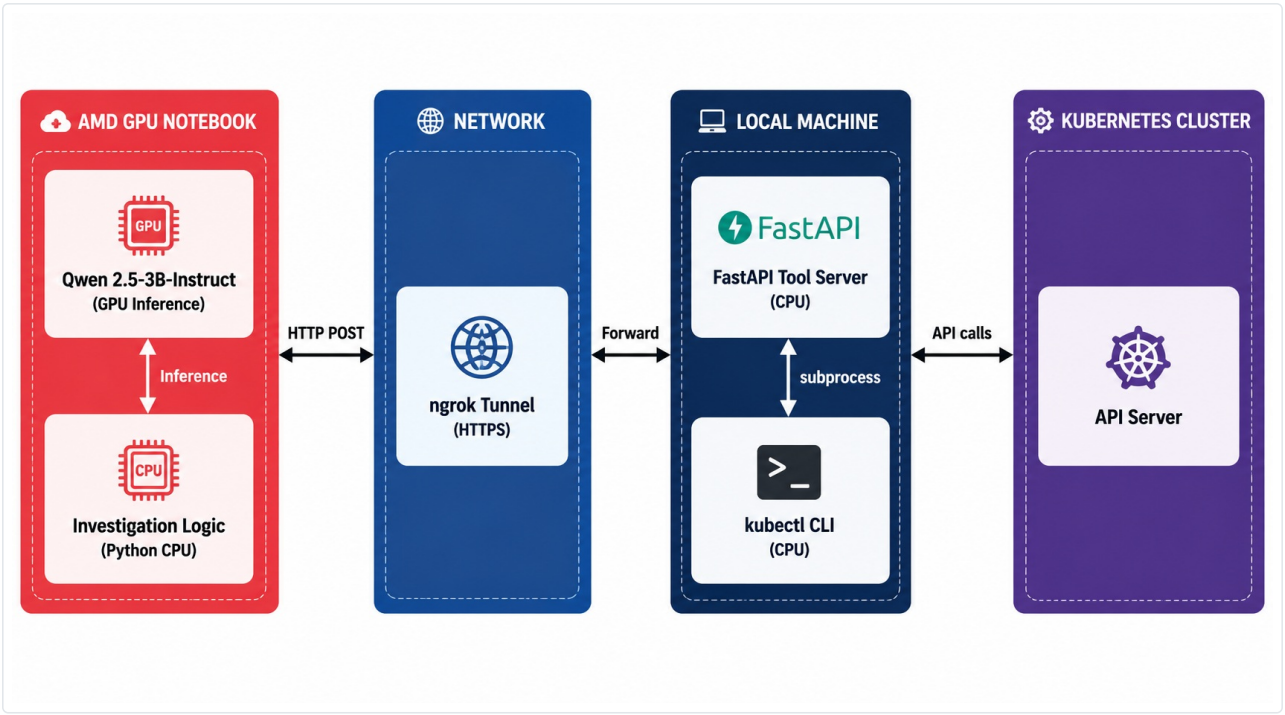
```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_NAME = "Qwen/Qwen2.5-3B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype="auto",
    device_map="auto"
)
```

The `device_map="auto"` parameter ensures the model is loaded onto available GPU memory, leveraging AMD Radeon GPU acceleration for tensor operations during inference.

10.2 Hybrid Deployment Architecture



Hybrid deployment architecture diagram showing the four-stage pipeline from AMD GPU Notebook through Network, Local Machine, to Kubernetes Cluster

10.3 Why This Architecture Was Selected

The Kubernetes cluster and its tools (`kubectl`) must remain on the local machine — the AMD notebook environment does not have access to the user's cluster credentials, kubeconfig, or network. Conversely, the Qwen model benefits from GPU acceleration available in the AMD notebook environment. The ngrok tunnel bridges these two environments cleanly:

1. **Security**: Cluster credentials never leave the local machine.
2. **Performance**: LLM inference runs on GPU-accelerated hardware.
3. **Modularity**: The tool server and the reasoning engine are independently deployable and testable.
4. **Portability**: The same FastAPI tool server can serve any notebook, cloud GPU instance, or inference endpoint.

10.4 GPU vs. CPU Execution Breakdown

Component	Execution Target	Rationale
Model loading (`AutoModelForCausalLM.from_pretrained`)	AMD GPU	Model weights (~6 GB) loaded into GPU VRAM
Tokenization (`tokenizer.apply_chat_template`)	CPU	Tokenization is a lightweight CPU operation
Tensor transfer (`.to(model.device)`)	GPU	Input tokens moved to GPU memory for inference
Inference (`model.generate()`)	AMD GPU	Matrix multiplications, attention computation, token generation — all GPU-accelerated
Token decoding (`tokenizer.decode`)	CPU	Lightweight string conversion
Investigation orchestration (investigate.py, tools.py)	CPU	Control flow, JSON parsing, HTTP requests
Deterministic pattern matching (regex)	CPU	Simple string matching
FastAPI tool server	CPU (local machine)	HTTP server processing

FastAPI tool server	CPU (local machine)	HTTP server processing
kubectl execution	CPU (local machine)	Kubernetes API interaction via subprocess

10.5 How AMD GPU Acceleration Benefits Reasoning

The Qwen 2.5-3B-Instruct model contains approximately 3 billion parameters. During each investigation iteration, the model processes a prompt containing the full workload description, all accumulated evidence, and the available tools schema. This prompt can reach 2,000–4,000 tokens in length. The model then generates a structured JSON response of 200–500 tokens.

AMD GPU acceleration benefits this workload in two key ways:

1. **Batch matrix multiplication**: The self-attention mechanism requires $O(n^2)$ computation over the input sequence. GPU parallelism reduces this from minutes (on CPU) to seconds.
2. **Token generation throughput**: Autoregressive generation requires one forward pass per output token. GPU acceleration ensures each forward pass completes in milliseconds rather than seconds, enabling responsive investigation cycles.

10.6 Modularity and Scalability Benefits

Separating inference from Kubernetes tool execution provides several engineering advantages:

- **Independent scaling**: The inference layer can be scaled to more powerful GPUs without touching the tool server. The tool server can be replicated across multiple clusters without touching the inference layer.
- **Model swappability**: The Qwen model can be replaced with any compatible model (larger Qwen variants, Llama, Mistral) by changing a single variable — the tool server is model-agnostic.
- **Multi-cluster support**: Multiple FastAPI tool servers (each connected to a different cluster) can be served by a single inference endpoint.
- **Testing in isolation**: The tool server can be tested independently with curl; the inference layer can be tested with mocked tool responses.

---

11. Future Work

The following enhancements represent realistic extensions of the implemented architecture:

Enhancement	Description	Architectural Impact
Multi-Cluster Support	Deploy multiple FastAPI tool servers connected to different clusters; the inference layer routes investigations by cluster ID.	Add cluster routing to tool dispatcher; no inference changes.
Additional Diagnostic Tools	Extend `kubernetes_tools.py` with tools for NetworkPolicies, Ingress, CertificateSigningRequests, CRDs, and Istio/Envoy sidecar diagnostics.	Add new functions to `kubernetes_tools.py` and entries to `AVAILABLE_TOOLS`.
Historical Incident Analysis	Store investigation results in a database to enable pattern recognition across incidents over time.	Add a persistence layer; the evidence store currently exists only in memory.
Prometheus/Grafana Integration	Add tools that query Prometheus metrics (CPU usage, memory pressure, request latency) to supplement kubectl-based evidence with time-series data.	Add new tool functions that query Prometheus HTTP API instead of kubectl.
RAG over Kubernetes	Implement retrieval-augmented	Add a vector store and retrieval step before each LLM

RAG over Kubernetes Documentation	Implement retrieval-augmented generation using official Kubernetes documentation to improve the LLM's reasoning about unfamiliar error conditions.	Add a vector store and retrieval step before each LLM call.
Multi-Agent Collaboration	Decompose the monolithic investigation into specialized sub-agents (network agent, storage agent, configuration agent) that collaborate on complex, multi-root-cause failures.	Significant architectural change: agent communication protocol, result aggregation.
Root Cause Ranking	When multiple hypotheses survive evidence elimination, rank them by likelihood using historical frequency data.	Extend the output schema; add a ranking step between investigation and reporting.

---

12. Conclusion

12.1 Technical Contributions

DevDas presents a complete, working implementation of an autonomous Kubernetes SRE investigation agent. The system's technical contributions include:

1. **A novel evidence-driven investigation loop** that mirrors the cognitive process of experienced SREs — forming hypotheses, selecting the most informative diagnostic tool, gathering evidence, and eliminating hypotheses until a confident root cause is reached.
2. **A hybrid deterministic-probabilistic architecture** where known failure signatures are resolved instantly via regex matching, while novel failures are investigated through LLM reasoning.
3. **A clean separation of inference and execution** that enables GPU-accelerated AI reasoning to operate on hardware physically separated from the Kubernetes cluster.
4. **17 purpose-built diagnostic tools** that provide comprehensive coverage across the Kubernetes resource model.
5. **A robust orchestration layer** with multiple termination guarantees (max iterations, tool exhaustion, duplicate detection, deterministic shortcuts) ensuring the agent always converges.

12.2 Innovation

DevDas innovates at the intersection of AI and infrastructure operations by demonstrating that:

- A 3B-parameter model, when properly prompted and constrained, can conduct systematic investigations that previously required senior engineering expertise.
- The one-tool-at-a-time constraint produces more reliable and interpretable investigations than unconstrained tool-calling approaches.
- Deterministic pattern matching and LLM reasoning are complementary — not competing — approaches that, combined, yield both speed and generality.

12.3 Practical Usefulness

DevDas is immediately useful for:

- **Reducing MTTR**: Automated first-pass investigations produce actionable diagnoses in minutes, not hours.
- **Augmenting on-call engineers**: The structured investigation reports serve as a starting point for human review, rather than requiring engineers to start from scratch.
- **Democratizing Kubernetes troubleshooting**: Junior engineers and students can learn SRE investigation methodology by observing the agent's step-by-step reasoning.
- **Providing off-hours coverage**: The agent can investigate incidents autonomously during periods when human SREs are unavailable.

12.4 Benefits to Kubernetes Operations

DevDas represents a step toward the vision of self-healing infrastructure — systems that not only detect failures but understand their causes and recommend (or execute) repairs. By combining the structured reliability of deterministic engineering with the adaptive reasoning of language models, DevDas demonstrates that autonomous infrastructure investigation is both feasible and practical with current AI technology.

> *This document was prepared for submission to the AMD AI Hackathon. All technical claims are derived from the implemented source code and Jupyter notebook. Features described as "Future Work" are explicitly designated as unimplemented extensions.*

Project Repository Components:

File	Purpose	Lines of Code
`kubernetes_tools.py`	kubectl subprocess wrappers (17 tools + discovery)	451
`investigate.py`	Investigation orchestrator + deterministic matcher	354
`tools.py`	Tool schema definitions + dispatcher	326
`system_prompt.py`	LLM system prompt (SRE persona, rules, output schema)	164
`report.py`	Cluster Investigation Report generator	160
`tools_server.py`	FastAPI tool server (HTTP API for remote tool execution)	79
`discovery.py`	Pure-Python cluster scan	55
`ask_llm.py`	LLM inference wrapper	37
`evidence.py`	Evidence store factory	16
`llm.py`	Model configuration	19
`devdas.ipynb`	Consolidated hackathon notebook (Qwen + ngrok)	~800 cells
Total		**~1,661+ LoC**